30-Day Java
Basic → Strong Foundation Plan

A structured, day-by-day roadmap to go from Java basics to a solid, interview-ready foundation — built around daily learning, coding, problem-solving, and one final capstone project.

Week 1	Java Fundamentals	Days 1–7
Week 2	Loops, Arrays & Strings	Days 8–14
Week 3	Object-Oriented Programming (Key)	Days 15–21
Week 4	Core Java Foundation & Final Project	Days 22–30

Daily rhythm: 1 hr Learn · 1 hr Code · 1 hr Problems · 30 min Revision (~3–3.5 hrs/day)

WEEK 1 — Java Fundamentals

Set up your environment and master the absolute basics of the language.

Day 1 — Java & Setup

Learn / Understand

- What is Java, and how it differs from other languages
- JDK vs JRE vs JVM — what each one actually does
- How Java code is compiled and executed

Do

- Install JDK and IntelliJ IDEA or Eclipse
- Learn class, main(), and System.out.println()
- Understand the difference between compiling and running code

Practice

- Write and run your first 5 small programs

Day 2 — Variables & Data Types

Learn / Understand

- Variables and how to declare them
- Primitive data types: int, float, double, char, boolean, long
- Non-primitive (reference) types
- Type casting
- The final keyword

Practice

- Store and print student details
- Build a simple calculator
- Convert temperature between units

Day 3 — Operators

Learn / Understand

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment / decrement operators
- Ternary operator
- Operator precedence

Practice

- Write and evaluate at least 15 different expressions

Day 4 — Input & Output

Learn / Understand

- Using the Scanner class
- Taking integer, string, and double input
- Reading multiple inputs in one program

Practice

- Calculate student marks
- Calculate salary
- Calculate an electricity bill
- Calculate simple interest

Day 5 — if / else

Learn / Understand

- if statement
- if-else statement
- else-if ladder
- Nested if statements
- Writing clear conditions

Practice

- Find the largest of 3 numbers
- Check even or odd
- Check positive or negative
- Build a grade calculator
- Check voting eligibility

Day 6 — switch

Learn / Understand

- switch statement
- case labels
- break
- default case
- Basics of the modern switch syntax

Build

Build a simple calculator using switch statements.

Day 7 — Revision + Mini Project

Don't learn new topics today — consolidate everything from Days 1–6.

Do

- Solve 20 basic problems
- Revise Days 1–6 thoroughly

Build

Build a Student Grade Calculator.

WEEK 2 — Loops, Arrays & Strings

Learn to repeat logic, work with collections of data, and manipulate text.

Day 8 — for Loop

Learn / Understand

- for loop syntax
- Nested for loops
- Counter logic

Practice

- Print numbers 1–10
- Print multiplication tables
- Sum of numbers
- Print even numbers
- Calculate factorial

Day 9 — while & do-while

Learn / Understand

- while loop
- do-while loop
- The difference between the two loop types

Practice

- Reverse a number
- Count digits in a number
- Sum of digits
- Check if a number is a palindrome

Day 10 — Patterns

Don't skip patterns — they build real understanding of nested loops.

Practice

- Star patterns: *, **, ***, ****, *****
- Number patterns: 1 / 12 / 123 / 1234
- Pyramid and other nested-loop patterns

Day 11 — Arrays

Learn / Understand

- What an array is
- Declaration and initialization
- Indexing
- Traversing an array
- Updating values

Practice

- Find the largest element
- Find the smallest element
- Sum of elements
- Average of elements
- Search for an element

Day 12 — Arrays — Advanced

Learn / Understand

- Reversing an array
- Copying an array
- Basics of sorting
- Finding duplicate elements
- Finding the second-largest element

Practice

- Solve 10–15 array problems covering the topics above

Day 13 — Strings

Learn / Understand

- The String class
- length()
- charAt()
- equals()
- substring()
- toUpperCase() / toLowerCase()
- indexOf()

Understand

- The difference between == and .equals() for comparing strings

Day 14 — Strings + Week 2 Revision

Practice

- Reverse a string
- Check if a string is a palindrome
- Count vowels
- Count characters
- Remove spaces
- Find duplicate characters

Build

Mini project — ATM Console Application.

WEEK 3 — Object-Oriented Programming — Key Week

The most important week — everything in real Java development builds on OOP.

Day 15 — Classes & Objects

A class is a blueprint; an object is an actual instance created from it.

Learn / Understand

- Class
- Object
- Fields
- Methods
- Creating objects
- Calling methods

Build

- Student class
- Car class
- BankAccount class
- Employee class

Day 16 — Constructors

Learn / Understand

- Default constructor
- Parameterized constructor
- Constructor overloading
- The this keyword

Practice

- Create Student, Employee, BankAccount, and Product classes using constructors

Day 17 — Methods

Learn / Understand

- Method declaration
- Parameters
- Return values
- void methods
- Method overloading
- Passing values to methods

Practice

- Write 15 different methods

Day 18 — Encapsulation

Learn / Understand

- private, public, protected access
- Getters and setters
- Why variables should often be kept private

Build

Bank Account class using encapsulation.

Day 19 — Inheritance

Learn / Understand

- Parent class and child class
- extends keyword
- super keyword
- Method overriding

Note: Example hierarchy: Vehicle → Car → ElectricCar

Day 20 — Polymorphism · KEY TOPIC

This concept is very important for backend development.

Learn / Understand

- Compile-time polymorphism (method overloading)
- Runtime polymorphism (method overriding)
- Upcasting, e.g. Animal a = new Dog();

Day 21 — Abstraction + Interface

Learn / Understand

- Abstract class
- Abstract method
- Interface
- implements keyword
- The difference between an abstract class and an interface

Build

Payment system: Payment → UPI, CreditCard, Cash

WEEK 4 — Core Java Foundation

Round out your foundation with the tools and concepts used in real projects and interviews.

Day 22 — Access Modifiers + Packages

Learn / Understand

- public, private, protected, default access
- Packages
- The import keyword
- How a typical Java project is structured

Day 23 — static, final, this, super

These four keywords come up constantly in interviews.

Learn / Understand

- static variable, static method, static block
- final variable, final method, final class
- this keyword
- super keyword

Day 24 — Exception Handling · KEY TOPIC

Learn / Understand

- try, catch, finally
- throw and throws
- Checked vs unchecked exceptions
- Custom exceptions

Practice

- Bank withdrawal program that throws an insufficient-balance exception

Day 25 — Collections — Introduction

Learn / Understand

- ArrayList: add, remove, get, set, size, contains

Then Learn

- HashSet
- HashMap

Note: Hierarchy: Collection → List → ArrayList

Day 26 — Collections — Practice

Practice

- ArrayList: 10 problems
- HashSet: duplicate removal
- HashMap: frequency counting (e.g. apple → 3, banana → 2, orange → 1)

Day 27 — File Handling + Java I/O

Don't spend too much time here — this is foundation-level knowledge.

Learn / Understand

- File
- FileReader / FileWriter
- BufferedReader / BufferedWriter

Build

Store student information in a file.

Day 28 — Java 8 Basics · KEY TOPIC

Learn / Understand

- Lambda expressions
- Functional interfaces
- forEach
- Stream basics: filter, map, collect

Note: Example: take a list of students → filter marks > 60 → display students. You don't need advanced Streams yet.

Day 29 — Complete Core Java Revision

Revise everything end-to-end, in order:

Learn / Understand

- Java basics → Variables → Operators → Conditions → Loops → Arrays → Strings
- → Methods → OOP → Exception Handling → Collections → Java 8

Practice

- Solve 30 coding problems covering all topics above

Day 30 — Final Java Foundation Project · KEY TOPIC

Build a Console-Based Banking Management System — this project forces you to use nearly everything you've learned.

Include

- **User:** create account, login, view profile
- **Bank Account:** deposit, withdraw, balance, transaction history
- **OOP:** classes, objects, encapsulation, inheritance, polymorphism, abstraction, interfaces
- **Collections:** ArrayList, HashMap
- **Exception Handling:** invalid amount, insufficient balance, invalid account, wrong login

How You Should Study Every Day

Avoid marathon video-watching sessions. Instead, follow this daily pattern:

1 hour	**Learn**	Understand the concept — read, watch, take notes.
1 hour	**Code**	Type out the examples yourself, line by line.
1 hour	**Problems**	Solve problems on your own, without looking at solutions.
30 min	**Revision**	Write down what you learned in your own words.

Total: approximately **3–3.5 hours a day** is enough — if you stay consistent.